

计算机图形学原理

-----教程7

李瑞辉

大纲

- 多纹理
- 纹理映射效果与光照的结合
- 作业 2 概述

多纹理

纹理单位

使用 `glUniform1i` 我们实际上可以为纹理采样器分配一个位置值，这样我们就可以在片段着色器中一次设置多个纹理。纹理的这个位置通常被称为纹理单元。纹理的默认纹理单元是 `0`，这是默认的活动纹理单元，所以我们不需要在上一节中分配位置

纹理单元的主要目的是允许我们在着色器中使用多于 `1` 个纹理。通过将纹理单元分配给采样器，我们可以在之后一次绑定到多个纹理 我们首先激活相应的纹理单元。

```
glActiveTexture (GL_TEXTURE0); //在e之前首先激活纹理单元 捆绑  
glBindTexture (GL_TEXTURE_2D, Texture);
```

激活纹理单元后，随后的`glBindTexture`调用会将该纹理绑定到当前活动的纹理单元。纹理单元`GL_TEXTURE0`始终默认激活，因此我们在使用`glBindTexture`时不必在前面的示例中激活任何纹理单元。

多纹理

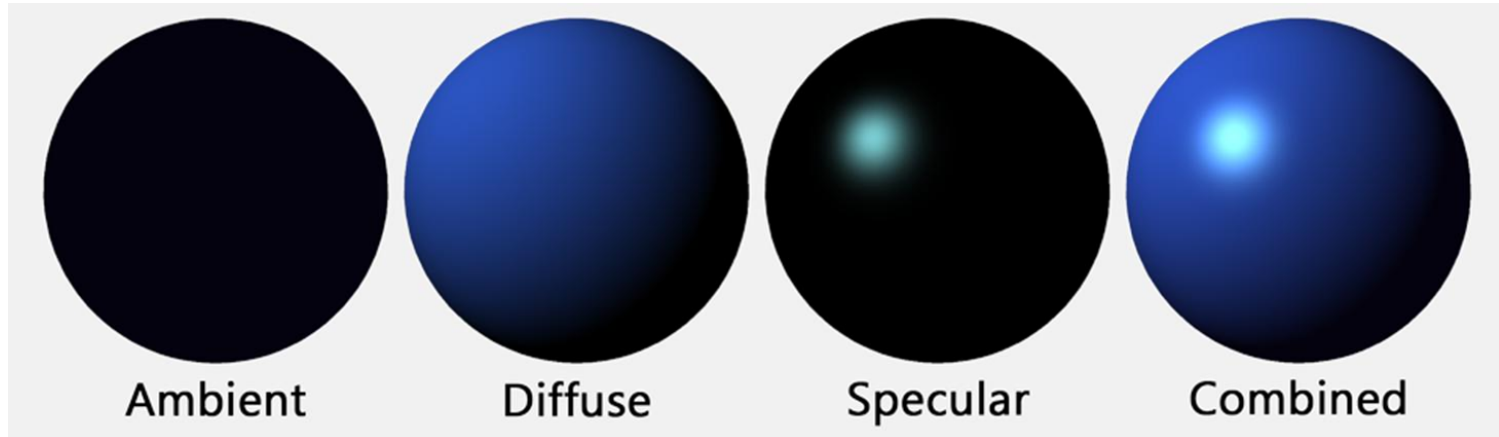
- “ **sendDataToOpenGL ()** ” 中加载多个纹理

```
Texture[0] = loadTexture("snow_01.jpg");  
Texture[1] = loadTexture("penguin_01.png");
```

- “ **paintGL (void)** ” 绘制之前绑定纹理

```
//First model  
glBindVertexArray(VertexArrayID[0]); //bind model  
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, Texture[0]); //bind texture  
  
glUniform1i(TextureID, 0); //We also have to tell OpenGL to which texture unit  
each shader sampler belongs to  
glDrawElements(GL_TRIANGLES, obj[0].indices.size(),  
               GL_UNSIGNED_INT, 0);  
  
...  
//Second model (same as the first one)  
...  
glBindTexture(GL_TEXTURE_2D, Texture[1]);  
...  
glDrawElements(GL_TRIANGLES, obj[1].indices.size(),  
               GL_UNSIGNED_INT, 0);  
  
...
```

召回 - 浅色



// Diffuse

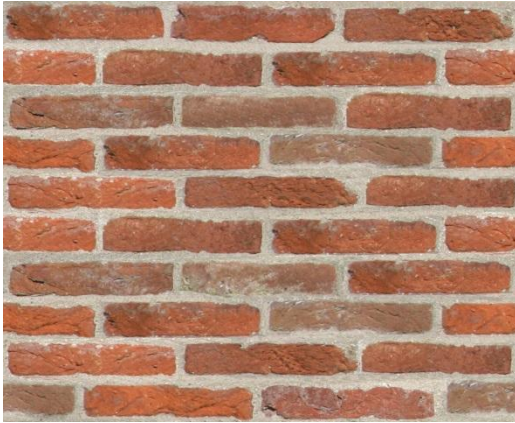
```
vec3 lightVectorWorld = normalize(lightPositionWorld - vertexPositionWorld);  
float brightness = dot(lightVectorWorld, normalize(normalWorld));  
vec4 diffuseLight = vec4(brightness, brightness, brightness, 1.0);
```

// Specular

```
vec3 reflectedLightVectorWorld = reflect(-lightVectorWorld, normalWorld);  
vec3 eyeVectorWorld = normalize(eyePositionWorld - vertexPositionWorld);  
float s = clamp(dot(reflectedLightVectorWorld, eyeVectorWorld), 0, 1);  
s = pow(s, 50);  
vec4 specularLight = vec4(s, 0, 0, 1);
```

```
daColor = ambientLight + clamp(diffuseLight, 0, 1) + specularLight;
```

召回 - 质地



```
in vec2 UV;  
  
uniform sampler2D myTextureSampler;  
  
int main()  
{  
    vec3 texture = texture( myTextureSampler, UV ).rgb;  
}
```

材质颜色

- 材料的颜色取决于模型反射的入射R、G、B光的百分比
- 反射照明亮度取决于入射照明亮度和模型反射率。
- 材质还包含不同种类的颜色，包括环境色、漫反射色和镜面反射色
- 确定材料的环境反射率、漫反射率和镜面反射率（值介于 0 和 1 之间）

材质颜色

- 对于光，RGB 数字对应于每种颜色的全强度百分比

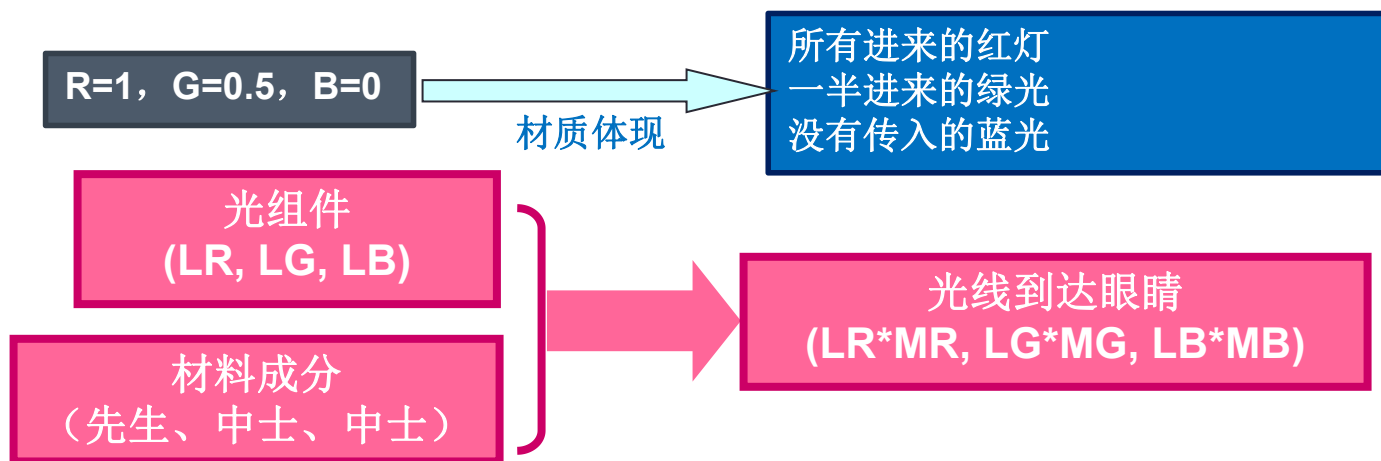
R=1, G=1, B=1

白色的

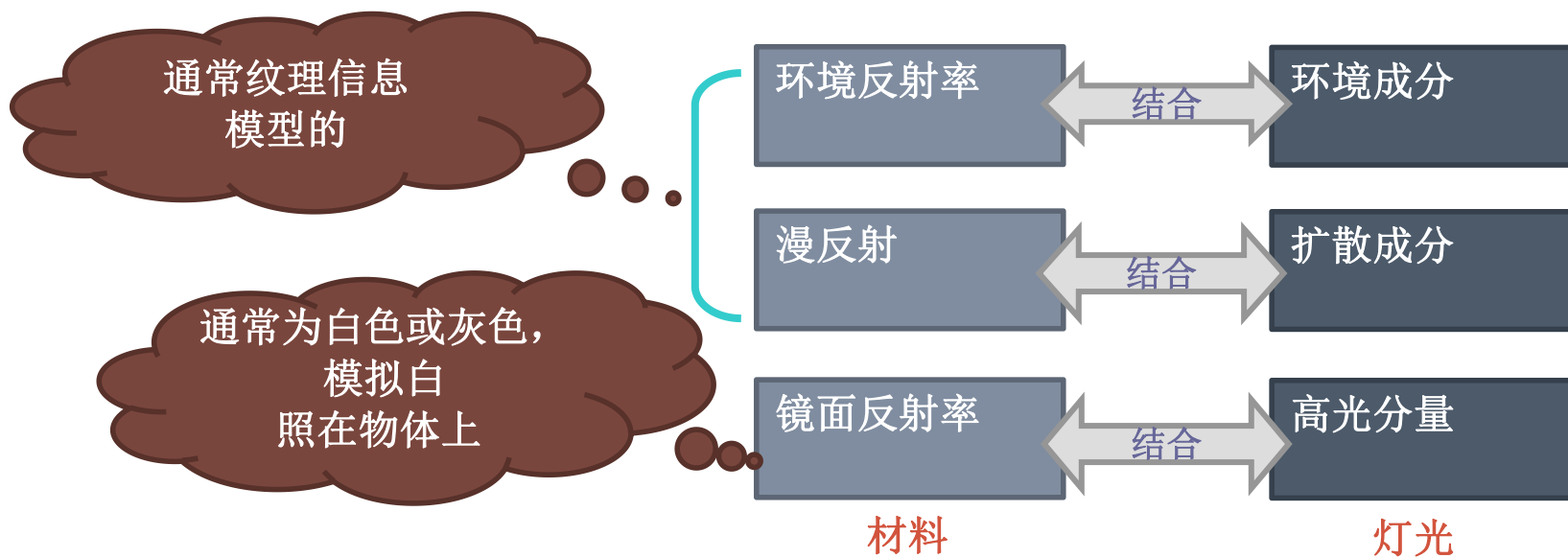
R=0.5, G=0.5, B=0.5

出现灰色

- 对于材质，RGB 数字对应于入射光中 RGB 分量的反射比例



材质颜色



灯光和材质的 RGB 值

- 示例代码：每个光组件的材质颜色

```
vec3 MaterialAmbientColor = 纹理 ( myTextureSampler , UV ) 。 RGB ;  
vec3 MaterialDiffuseColor = 纹理 ( myTextureSampler , UV ) 。 RGB ;  
vec3 MaterialSpecularColor = vec3(0.3,0.3,0.3); // 或者镜面纹理
```

- 示例代码：最终颜色

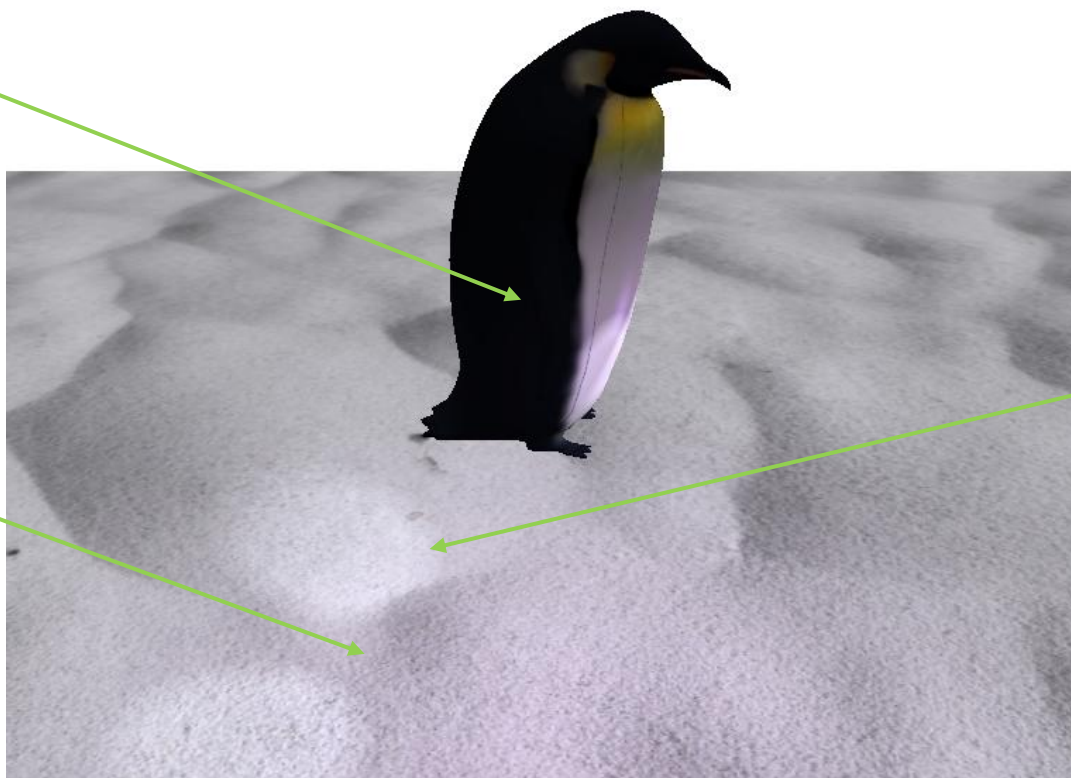
```
// calculates the color when using a directional light.  
vec3 CalcDirLight(DirLight light, vec3 normal, vec3 viewDir)  
{  
    vec3 lightDir = normalize(-light.direction);  
    // diffuse shading  
    float diff = max(dot(normal, lightDir), 0.0);  
    // specular shading  
    vec3 reflectDir = reflect(-lightDir, normal);  
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);  
    // combine results  
    vec3 ambient = light.ambient * vec3(texture(material.ambient, textCoord));  
    vec3 diffuse = light.intensity * light.diffuse * diff * vec3(texture(material.diffuse, textCoord));  
    vec3 specular = light.intensity * light.specular * spec * vec3(texture(material.specular, textCoord));  
    return (ambient + diffuse + specular);  
}
```

作业 2

作业 2:

由 .obj 文件加载的复杂企鹅模型

由 .obj 文件加载的雪地面



灯光

图 1 演示程序绘制的场景。



作业 2

演示



作业 2

基本要求:

- 加载复杂对象
- 纹理贴图
- 灯光
- 互动活动和动画

作业 2

基本要求:

□ 加载复杂对象

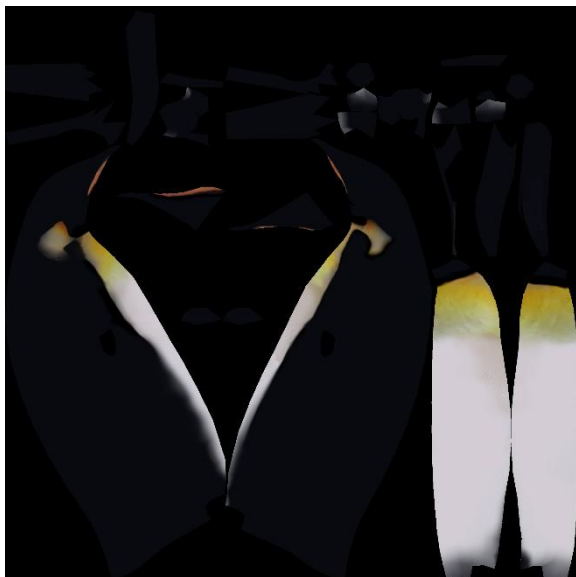
- 我们提供的函数'`Model loadOBJ ()`'，或者Open Asset Import Library来加载至少两个复杂模型。
- 我们在演示程序中提供了模型，即`snow.obj` 和`penguin.obj`。我们鼓励您从Internet 下载其他 `.obj` 文件或使用 Blender 来设计您的对象。

作业 2

基本要求:

□ 纹理贴图

- 将不同的纹理映射到两个模型，即演示程序中的猫和地平面。
- 我们在demo程序中也提供了模型的贴图，即penguin_01.png、penguin_02.png和snow_01.jpg、snow_02.jpg。我们也鼓励您从Internet下载其他纹理。



企鹅_01.png



企鹅_02.png

作业 2



snow_01.jpg



snow_02.jpg

作业 2

基本要求：

- 带键盘控制的纹理开关
- 为 penguin.obj 应用不同的纹理
- 如 penguin_01 – penguin_02.png



作业 2

- 雪.obj应用不同的纹理
- 比如snow_01 – snow_02.jpg



作业 2

提示：

- struct keyboardcontroller

```
{
```

```
    int theme_penguin;
```

```
    int theme_snow;
```

```
    ...
```

```
};
```

```
keyboardcontroller keyboardCtl;
```

- penguin/snowTexture[keyboardCtl.theme...].bind(0) -----In Texture.cpp

作业 2

提示:

- `void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods)`

```
{
```

```
    // Sets the Keyboard callback for the current window.
```

```
    if (key == GLFW_KEY_1 && action == GLFW_PRESS)
```

```
    {
```

```
        keyboardCtl.theme_ = ...;
```

```
    }
```

作业 2

基本要求：

□ 灯光

- 应至少用两个光源照明。
- 一个应该是环境光。（定向光）
- 对于其他光源，您可以自行决定其位置和颜色。

作业 2

基本要求：

□ 互动活动和动画

灯光控制

按“w”键和“s”键分别增加和减少定向光的亮度。

■ 查看控件

通过鼠标控制摄像头的视角，即：

当左键点击，鼠标上下移动时，你看到的整个场景也会随之上下移动。

作业 2

提示: Increase intensity

- struct keyboardcontroller

{

 float intensity = 1.0f; ...

};
- keyboardcontroller keyboardCtl;
shader.setFloat("dirLight.intensity", keyboardCtl.intensity);
- if (key == GLFW_KEY_W && action == GLFW_PRESS)
{

 if (keyboardCtl.intensity < some threshold)
 keyboardCtl.intensity *= some scale;

}

作业 2

提示： 改变相机的位置

```
void Camera::ProcessMouseMovement_Left(float xoffset, float yoffset,
GLboolean constrainPitch)
{
    // Processes input received from a mouse input system. Expects the
offset value in both the x and y direction.

    xoffset *= MouseSensitivity;

    yoffset *= MouseSensitivity;

    Yaw += xoffset;

    Pitch += yoffset;

    // Update Front, Right and Up Vectors using the updated Euler
angles

    updateCameraVectors();
}
```

作业 2

提示：改变相机的位置

- ```
struct MouseController {
 bool LEFT_BUTTON = false;
 bool RIGHT_BUTTON = false;
 double MOUSE_Clickx = 0.0, MOUSE_Clicky = 0.0;
 double MOUSE_X = 0.0, MOUSE_Y = 0.0;
 int click_time = glfwGetTime();
};
```
- ```
MouseController mouseCtl;
```

作业 2

提示： 改变相机的位置

```
void mouse_button_callback(GLFWwindow* window, int button, int action, int mods) { ...
```

```
//Capture the mouse button click action, can be used to record the cursor position when the mouse clicked
```

```
    if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_PRESS)
```

```
        { ... }
```

```
    if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_RELEASE)
```

```
        { ... }}
```

```
void cursor_position_callback(GLFWwindow* window, double x, double y {
```

```
// Reports mouse-motion while a button is held, can be used to record the cursor position)
```

```
//to get real time cursor position
```

```
...
```

```
    if (mouseCtl.LEFT_BUTTON) //get the state of mouse
```

```
        {...}
```

```
...}
```

作业 2

基本要求：

- 对象控制

按方向键“↑↓←→”可以控制一个复杂模型在地平面上的移动。具体来说，“↑↓”分别表示向前和向后移动。“←→”分别表示左右旋转。

作业 2

提示：

改变对应的模型矩阵

作业 2

奖励任务：

- 加载更复杂的模型并将其他纹理映射到它们上以形成您喜欢的有意义的场景。(10%)

Pointlight、Spotlight 等的组合(10%)

- 复杂模型的阴影映射。(10%)
- 绘制点或线来追踪其中一个复杂模型的运动。(10%)
- 任何其他有趣的效果。(10%)

作业 2

提交编程作业的指南：

- 我们已经提供了任务 2 的框架代码。
- 在作业 2 中，修改提供的 `main.cpp` & `VertexShaderCode.glsl` & `FragmentShaderCode.glsl` 。
- 额外的，鼓励使用 Camera Class 的 `cpp` 或 `.h` 文件。
- 您可以创建额外的 `.glsl` 文件，如果你想实现阴影贴图技术。
- 在 `main.cpp` 中输入你的全名和学号。

作业 2

提交：

- 源代码文件（即 `main.cpp` & `VertexShaderCode.glsl` & `FragmentShaderCode.glsl`）；
- 可执行文件（例如，`openGL.exe`）；
- 自述文件；
- 您创建或下载的 `.obj` 文件；
- 您下载的图像纹理文件。

下一教程

➤ 阴影贴图

谢谢